

AI Remote Control System

Complete Architecture & Full File Structure Reference

React Native

FastAPI

WebSockets

LangChain / AI Agents

Python Desktop Agent

This project turns a phone into an AI-native command center for a PC — live screen streaming, remote terminal, file management, app launching, system monitoring, and an autonomous AI agent that can plan and execute multi-step tasks on the desktop (open apps, run code, fix errors, report back). It is built as three cooperating services: a **React Native** mobile app, a **FastAPI** backend, and a **Python desktop agent** running on the target PC, connected over authenticated, encrypted WebSockets.

Table of Contents

- 1. System Overview & Goals
- 2. High-Level Architecture
- 3. Tech Stack
- 4. Complete Repository File Structure
- 5. Mobile App Architecture
- 6. Backend Architecture
- 7. Desktop Agent Architecture
- 8. Backend API Reference
- 9. WebSocket Event Reference
- 10. Database Schema
- 11. AI Integration Layer
- 12. Security Architecture
- 13. Streaming Pipeline
- 14. Getting Started
- 15. Development Roadmap & Scalability
- 16. Future Expansion Ideas
- 17. License

1. System Overview & Goals

Goal: a secure, AI-native remote-control platform combining classic remote-desktop functionality (screen share, mouse/keyboard, terminal, files) with an autonomous AI layer that interprets natural-language instructions, plans steps, executes tools on the PC, verifies results, and retries on failure.

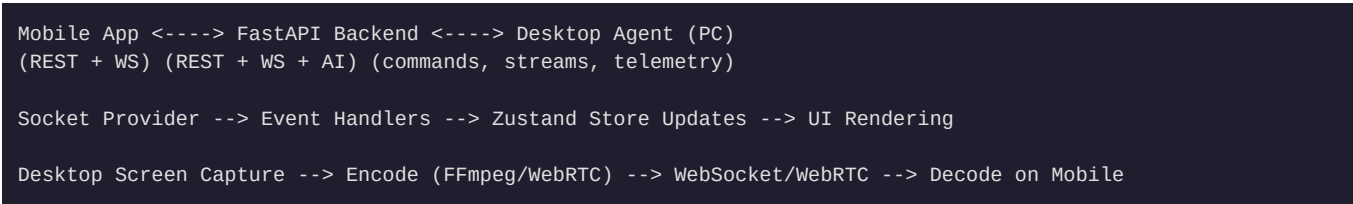
- Live desktop screen streaming with touch-based mouse/keyboard control
- Remote terminal with real-time output streaming and AI-assisted debugging
- Full file manager — browse, upload, download, preview, search PC files
- AI Assistant — chat interface, voice commands, autonomous task workflows
- App launcher and installed/running-app monitor
- Live system monitoring — CPU, RAM, GPU, network, temperature, battery, processes
- JWT auth, device pairing, encrypted WebSocket/TLS communication, audit logging

Design language (mobile app): dark futuristic UI, glassmorphism, neon cyan/purple/electric-blue accents, dashboard-first layout, smooth page transitions, live terminal-typing and streaming effects. Typography: Inter, Space Grotesk, JetBrains Mono. The product should feel like a cyberpunk developer OS — an AI command center / next-generation remote desktop platform.

2. High-Level Architecture

Layer	Responsibility	Core Tech
Mobile App	UI, device pairing, live control, chat, monitoring dashboards	React Native, Zustand, Expo, WebRTC
Backend API	Auth, routing, AI orchestration, WebSocket hub, persistence	FastAPI, PostgreSQL, Redis
Desktop Agent	Executes commands on the PC: mouse/keyboard, terminal, files, system info	Python, pyautogui, mss, psutil
AI Layer	Intent parsing, task planning, tool execution, memory	LangChain / AutoGen / CrewAI
Data Layer	Relational data, cache/pub-sub, vector memory, telemetry	PostgreSQL, Redis, Vector DB

Communication flow:



Why WebSockets: low latency, real-time bidirectional updates for terminal output, screen frames, device heartbeat, and mouse/keyboard sync. Redis Pub/Sub is the recommended path for scaling socket fan-out across multiple backend instances.

3. Tech Stack

Component	Technologies
Mobile Frontend	React Native, TypeScript, Expo, Zustand, NativeWind/Tailwind, react-native-webrtc
Backend	FastAPI, Pydantic, SQLAlchemy, PostgreSQL, Redis, WebSockets, JWT

Desktop Agent	Python, pyautogui (mouse/keyboard), mss (screenshots), psutil (monitoring), subprocess, watchdog
AI / Automation	LangChain, AutoGen, CrewAI, or Open Interpreter; vector embeddings for memory
Streaming	WebRTC / FFmpeg, adaptive bitrate, differential frame updates, GPU encoding
DevOps	Docker, Docker Compose, Kubernetes, GitHub Actions, Prometheus, Grafana, Sentry

4. Complete Repository File Structure

Monorepo layout. Each top-level service (mobile-app, backend, desktop-agent) is independently runnable and independently deployable; shared/ holds cross-service contracts and infrastructure/ holds all DevOps configs.

4.1 mobile-app/ (React Native Frontend)

```
mobile-app/
  .env · app.json · babel.config.js · tailwind.config.js · tsconfig.json · package.json
  assets/ fonts/, images/, animations/, splash.png
  src/
    App.tsx
    screens/
      auth/ SplashScreen, LoginScreen, RegisterScreen, DevicePairingScreen
      dashboard/ DashboardScreen, ConnectionStatusCard, PCStatusWidget,
        SystemMetricsWidget, RecentCommandsList
      connection/ ConnectionCenterScreen, DeviceListScreen, AddPCDeviceScreen,
        PCConnectionDetails, ConnectionHistoryScreen
      remote/ RemoteDesktopScreen, ScreenStreamViewer, TouchMouseControls,
        KeyboardOverlay, RemoteControlBar
      terminal/ TerminalScreen, TerminalOutput, CommandInput, CommandHistoryList,
        CodeEditorModal, ExecutionResultViewer
      files/ FileManagerScreen, FileBrowser, FileDownloadProgress,
        FileUploadModal, FilePreviewModal, FileSearchBar
      ai/ AIAssistantScreen, ChatInterface, VoiceCommandButton,
        TaskWorkflowBuilder, SuggestionChips, AIExecutionStatus
      apps/ AppLauncherScreen, InstalledAppsList, AppCategoryGrid,
        RunningAppsMonitor
      settings/ SettingsScreen, ProfileSettings, SecuritySettings,
        PCDeviceSettings, StreamingQualitySettings, NotificationSettings
      monitoring/ SystemMonitorScreen, CPUMetricsChart, RAMMetricsChart,
        NetworkSpeedMonitor, ActiveProcessesList
      components/
        common/ Button, Card, Input, Modal, LoadingSpinner, ToastNotification,
        ConnectionBadge
        connection/ PCConnectionCard, ConnectionStatusIndicator, PCOnOffStatus,
        PermissionRequestModal, QRCodePairing
        streaming/ VideoStreamPlayer, StreamQualitySelector, FPSMonitor
      services/
        api/ apiClient, auth.service, device.service, command.service,
        file.service, app.service, ai.service, stream.service
        websocket/ socketClient, socketEvents, connectionManager, terminalSocket,
        streamSocket, heartbeatService
        storage/ secureStorage, localStorage, fileCache
      store/ index, authStore, deviceStore, connectionStore, terminalStore,
        fileStore, streamStore, aiStore, appStore (Zustand)
      hooks/ useWebSocket, useAuth, useDeviceConnection, useTerminal,
        useFileTransfer, useScreenStream
      types/ auth, device, command, file, stream, ai (.types.ts)
      utils/ validators, formatters, encryption, constants
      navigation/ AppNavigator, AuthNavigator, MainTabNavigator, types, linking
    __tests__/ screens/, services/, store/
```

4.2 backend/ (FastAPI Backend)

```
backend/
  .env · .env.example · requirements.txt · requirements-dev.txt
  Dockerfile · docker-compose.yml · pyproject.toml
  app/
    main.py · config.py · dependencies.py
    api/
      router.py
```

```

■ ■ ■ ■ v1/ auth.py, devices.py, commands.py, files.py, apps.py,
■ ■ ■ ai.py, streaming.py, monitoring.py
■ ■ ■ ■ websocket/ connection_manager.py, terminal_handler.py, stream_handler.py,
■ ■ command_handler.py, heartbeat_handler.py
■ ■ ■ ■ core/ security.py, auth.py, exceptions.py, logging.py, rate_limiter.py
■ ■ ■ ■ models/ user.py, device.py, command.py, session.py, file_transfer.py,
■ ■ ai_memory.py
■ ■ ■ ■ schemas/ auth.py, device.py, command.py, file.py, ai.py, streaming.py
■ ■ ■ ■ services/ auth_service.py, device_service.py, command_service.py,
■ ■ file_service.py, app_launcher_service.py, terminal_service.py,
■ ■ streaming_service.py, monitoring_service.py
■ ■ ■ ■ ai/ orchestrator.py, intent_parser.py, task_planner.py,
■ ■ tool_executor.py, memory_manager.py, langchain_setup.py,
■ ■ prompts.py
■ ■ ■ ■ db/ database.py, redis_client.py, migrations/{versions/, env.py}
■ ■ ■ ■ utils/ validators.py, encryption.py, helpers.py, websocket_utils.py
■ ■ ■ ■ tests/ conftest.py, test_api/, test_services/, test_websocket/

```

4.3 desktop-agent/ (Python Desktop Agent)

```

desktop-agent/
■ ■ ■ ■ .env · requirements.txt · setup.py · pyproject.toml · Dockerfile
■ ■ ■ agent/
■ ■ ■ ■ main.py (entry point) · config.py · logger.py
■ ■ ■ ■ connection/ websocket_client.py, connection_manager.py, heartbeat.py,
■ ■ reconnection_handler.py, device_info.py
■ ■ ■ ■ automation/ mouse_controller.py, keyboard_controller.py, app_launcher.py,
■ ■ window_manager.py, hotkey_manager.py
■ ■ ■ ■ terminal/ command_executor.py, shell_handler.py, process_manager.py,
■ ■ output_streamer.py, code_runner.py
■ ■ ■ ■ streaming/ screen_capturer.py, frame_encoder.py, stream_sender.py,
■ ■ quality_optimizer.py, multi_monitor.py
■ ■ ■ ■ files/ file_manager.py, file_uploader.py, file_downloader.py,
■ ■ file_watcher.py, directory_browser.py
■ ■ ■ ■ system/ system_monitor.py, resource_tracker.py, process_monitor.py,
■ ■ network_monitor.py, power_manager.py
■ ■ ■ ■ security/ authenticator.py, permission_checker.py, command_sanitizer.py,
■ ■ encryption.py, audit_logger.py
■ ■ ■ ■ utils/ helpers.py, validators.py, constants.py
■ ■ ■ ■ installers/ windows_installer.ps1, mac_installer.sh, linux_installer.sh,
■ ■ permissions_setup.py
■ ■ ■ ■ tests/ test_automation/, test_terminal/, test_streaming/, test_files/

```

4.4 shared/, infrastructure/, docs/, .github/

```

shared/ # Cross-service shared code
■ ■ ■ ■ types/ events.ts, commands.ts, models.ts
■ ■ ■ ■ utils/ encryption.ts, validators.ts

infrastructure/ # DevOps & Deployment
■ ■ ■ ■ docker/ backend.Dockerfile, desktop-agent.Dockerfile, nginx.conf,
■ ■ docker-compose.prod.yml
■ ■ ■ ■ kubernetes/ deployment.yaml, service.yaml, ingress.yaml, configmap.yaml
■ ■ ■ ■ monitoring/ prometheus.yml, grafana-dashboard.json, alerts.yml
■ ■ ■ ■ scripts/ deploy.sh, backup.sh, healthcheck.sh

docs/ # Documentation
■ ■ ■ ■ API.md · WEBSOCKET_EVENTS.md · DEPLOYMENT.md · SECURITY.md · USER_GUIDE.md

.github/workflows/ # CI
■ ■ ■ ■ backend-ci.yml · mobile-ci.yml · desktop-agent-ci.yml

Root: .gitignore · README.md · LICENSE · docker-compose.yml

```

5. Mobile App Architecture

5.1 Navigation

- **Root Navigation:** Authentication Stack, Main App Stack
- **Main App Tabs:** Dashboard · Remote · Terminal · AI Assistant · Settings
- **Nested Navigators:** Device details, File manager, Monitoring pages, Stream controls

5.2 State Management (Zustand)

Store	Holds
authStore	user, token, permissions
deviceStore	connected devices, active device, device status
streamStore	video stream, stream quality, fullscreen state
terminalStore	logs, command history, active sessions
aiStore	chat history, AI status, workflows
notificationStore	alerts, system messages

5.3 Core Screen Functionality

Screen	Key Features
Authentication	Splash, login, register, device pairing
Main Dashboard	Connected devices, CPU/GPU usage, running apps, AI quick actions, recent commands
Remote Desktop	Live PC stream, touch mouse controls, keyboard overlay, fullscreen
Terminal	Command execution, real-time logs, syntax highlighting, AI code analysis
AI Assistant	ChatGPT-like interface, voice commands, suggested actions, autonomous workflows
File Manager	Browse PC files, upload/download, drag-and-drop interactions
Monitoring	System metrics, RAM usage, temperature, internet speed

5.4 Remote Desktop Controls & Streaming

- Tap = click · Two-finger drag = scroll/pan · Pinch = zoom · Long press = right click
- Overlay components: floating keyboard, FPS indicator, network quality, AI assistant overlay
- Streaming stack: react-native-webrtc + WebRTC signaling + adaptive quality
- Optimizations: frame buffering, hardware acceleration, compression handling, picture-in-picture

5.5 AI Assistant Frontend

- Streaming AI responses, tool execution previews, workflow builder, task timeline
- Voice commands, image understanding, code explanation, AI-generated automation
- Suggested components: ChatBubble, WorkflowCard, AgentStatus, AIReasoningPanel

5.6 Frontend Security & Performance

- JWT auth, secure token storage (expo-secure-store), biometric login (expo-local-authentication), encrypted WebSockets, device authorization
- Lazy loading, memoization, virtualized lists, WebSocket batching, debouncing, image caching
- Streaming: adaptive bitrate, frame skipping, dynamic resolution

5.7 Notifications

- Types: AI task completed, device offline, error alerts, stream disconnected, terminal finished
- Channels: push notifications, in-app banners, toast messages, real-time alerts

6. Backend Architecture

FastAPI service exposing REST endpoints for CRUD/auth and a WebSocket hub for real-time terminal, streaming, and AI events. Organized into `api/` (routes + websocket handlers), `core/` (security, auth, exceptions, logging, rate limiting), `models/` + `schemas/` (ORM + Pydantic), `services/` (business logic), `ai/` (agent orchestration), and `db/` (database + Redis + migrations).

6.1 Responsibilities by Module

Module	Responsibility
<code>api/v1/*</code>	REST routes: auth, devices, commands, files, apps, ai, streaming, monitoring
<code>api/websocket/*</code>	Connection manager, terminal/stream/command handlers, heartbeat handler
<code>core/*</code>	JWT security, auth dependencies, custom exceptions, structured logging, rate limiting
<code>models/*</code> + <code>schemas/*</code>	SQLAlchemy ORM models and Pydantic request/response schemas
<code>services/*</code>	Business logic per domain (auth, device, command, file, app launcher, terminal, streaming, monitoring)
<code>ai/*</code>	Orchestrator, intent parser, task planner, tool executor, memory manager, LangChain setup, prompts
<code>db/*</code>	Database session, Redis client, Alembic-style migrations

7. Desktop Agent Architecture

A lightweight Python process installed on the target PC. It authenticates to the backend, maintains a persistent WebSocket connection with heartbeat/reconnection handling, and executes incoming commands.

7.1 Core Responsibilities

- Receive remote commands from the backend over WebSocket
- Execute terminal instructions and stream output back in real time
- Capture screen frames and stream them (multi-monitor aware)
- Monitor system health — CPU, RAM, processes, network, power
- Control mouse and keyboard (automation module)
- Return logs/output and enforce local security checks before executing sensitive commands

7.2 Key Python Libraries

Library	Purpose
<code>pyautogui</code>	Keyboard & mouse control
<code>mss</code>	Fast screenshots / screen capture
<code>subprocess</code>	Command & process execution
<code>psutil</code>	System monitoring (CPU, RAM, processes, network)
<code>watchdog</code>	File system change tracking

7.3 Agent-Side Security

- Encrypted communication (TLS WebSockets)
- Device pairing with signed tokens
- Permission controls and command sanitization
- Local confirmation prompts for dangerous commands
- Sandboxed execution and audit logging

8. Backend API Reference

Domain	Endpoints
Auth	POST /auth/register · POST /auth/login · POST /auth/refresh · POST /auth/logout
Devices	GET /devices · POST /devices/pair · DELETE /devices/{id}
Commands	POST /commands/execute · GET /commands/history · POST /commands/terminal
Files	GET /files/list · POST /files/upload · GET /files/download
AI	POST /ai/chat · POST /ai/execute-task · GET /ai/history
Streaming	GET /stream/session · POST /stream/start · POST /stream/stop

9. WebSocket Event Reference

Event	Purpose
connect_device / disconnect_device	Device session lifecycle
client_connected / agent_connected	Mobile client and desktop agent handshakes
command_execute / command_output	Command dispatch and results
terminal_output	Live terminal stdout/stderr streaming
screen_frame / stream_frame	Live screen streaming frames
mouse_move / keyboard_input	Remote input control
ai_response / ai_progress	AI assistant streaming responses and progress
file_upload_progress	File transfer progress updates
system_alert	Device offline, error, or disconnect notifications
heartbeat / typing_status	Connection keep-alive and live activity indicators

Recommended architecture: **Socket Provider** → **Event Handlers** → **Zustand Store Updates** → **UI Rendering**. Use Redis Pub/Sub on the backend to fan events out across multiple server instances at scale.

10. Database Schema

Table	Key Fields
users	id, email, password_hash, created_at
devices	id, user_id, device_name, public_key, last_seen
commands	id, user_id, command, output, status, created_at
sessions	id, user_id, token, expires_at
ai_memory	id, user_id, context, embeddings

PostgreSQL is the system of record; Redis provides caching and pub/sub; a vector database backs AI memory (embeddings) for long-term context recall across sessions.

11. AI Integration Layer

- **Intent Parser** — converts natural language into structured tasks
- **Task Planner** — breaks complex requests into ordered execution steps
- **Tool Executor** — invokes desktop tools: terminal, browser, file manager, mouse/keyboard controller
- **Memory Manager** — stores past tasks, outputs, and user preferences as vector embeddings
- **Autonomous Agent Loop** — Observe → Plan → Execute → Verify → Retry

Recommended frameworks: LangChain, AutoGen, CrewAI, or Open Interpreter.

Example Workflow

```
User: "Open VS Code, run app.py, fix errors, and send output."

1. AI parses the instruction
2. Planner generates execution steps
3. Desktop agent launches VS Code
4. Terminal executes the Python file
5. AI reads the logs; if errors occur it analyzes the traceback,
   applies a fix, and re-runs the code
6. Final output is streamed back to the mobile app
```

12. Security Architecture

Area	Measures
Authentication	JWT access + refresh tokens, device authorization, biometric login on mobile
Transport	HTTPS, TLS-secured WebSockets, end-to-end encryption for sensitive payloads
Desktop Agent	Local confirmation for dangerous commands, sandboxed execution, restricted permissions, command sanitization
Auditing	Activity tracking, session auditing, full command history logging

13. Streaming Pipeline

```
Desktop Screen Capture (mss / multi-monitor)
--> Frame Encoder (FFmpeg / WebRTC, GPU encoding)
--> Stream Sender (WebSocket / WebRTC, adaptive bitrate)
--> Decode & Render on Mobile (react-native-webrtc)
```

- Performance: adaptive bitrate, differential frame updates, GPU encoding, compression
- Advanced: multi-monitor support, audio streaming, clipboard sync, picture-in-picture

14. Getting Started

Prerequisites

- Node.js 18+ and npm/yarn (mobile app)
- Python 3.10+ (backend & desktop agent)
- PostgreSQL and Redis (local or Docker)
- Docker & Docker Compose (recommended)

Quick Setup

```
# 1. Clone
git clone <repo-url> && cd ai-remote-control-system

# 2. Backend
cd backend && cp .env.example .env
pip install -r requirements.txt
uvicorn app.main:app --reload

# 3. Desktop Agent (run on the PC to be controlled)
cd ../desktop-agent
pip install -r requirements.txt
python agent/main.py

# 4. Mobile App
cd ../mobile-app
npm install
npx expo start

# Or, run backend + dependencies with Docker:
docker-compose up --build
```

Then pair the desktop agent with the mobile app via the in-app QR code pairing screen.

15. Development Roadmap & Scalability

15.1 Frontend Build Phases

Phase	Focus
1	Auth screens, navigation, dashboard
2	WebSockets, device management, terminal UI
3	Screen streaming, mouse controls
4	AI assistant, automation workflows
5	Advanced animations, performance tuning, production deployment

15.2 Full-System Timeline

Phase	Focus
Month 1	Authentication, backend APIs, mobile app scaffolding

Month 2	WebSockets, desktop agent, command execution
Month 3	Screen streaming, terminal integration
Month 4	AI assistant, task automation
Month 5	Security hardening, performance optimization
Month 6	Production deployment

15.3 Scalability Plan

- Phase 1 — single-user architecture
- Phase 2 — multi-device synchronization
- Phase 3 — distributed AI execution
- Phase 4 — cloud-hosted desktop agents
- Phase 5 — enterprise multi-user infrastructure

15.4 Deployment

- **Containers:** backend, redis, postgres, ai-service
 - **Hosting options:** AWS EC2, Railway, Render, DigitalOcean
 - **CI/CD:** GitHub Actions for builds, tests, automated deploys
 - **Monitoring:** Prometheus + Grafana dashboards, Sentry error tracking
-

16. Future Expansion Ideas

- Voice assistant integration
 - GPU monitoring & cloud compute management
 - Fully autonomous coding agent
 - Browser automation
 - Smart-home control hooks
 - Visual AI workflow builder
 - Local LLM execution (on-device inference)
 - Team collaboration mode
-

17. License

This project is provided under the license specified in LICENSE at the repository root. Review and adapt it before distribution.
